What Would Neural Shrimp Structures Let Us Do?

Dynamical Signatures, Operational Consequences, and a Testable Research Program

Authorship to be finalized by the human researcher

11 September 2026

Private exploratory working draft. Initial local pilots, evidence-linked figures, and proposed follow-up experiments. No neural-network shrimp or optimization benefit has yet been established by this project. This document is not a preregistration or publication-ready article.

Abstract

A conspicuous structure in a neural network's dynamics should have a dynamical consequence. Which consequences reach predictions, task accuracy, or compute cost is a separate, measurable question. Shrimp-shaped parameter windows organize stable periodic behavior and have been reported in small recurrent neural networks. We develop a program to test whether their geometry supplies useful control information: which intervention direction helps, how large a change is required, and how robust the destination is. An exact tied-weight toy yields alternating predictions and lower squared error for averaged predictions than for random-phase readout, with residual bias. Initial exploratory pilots verify attracting nontrivial cycles in 39 of 65 Hénon cells, but find 0 such cycles in 162 tiny-network training cells, with 107 unresolved and 55 operationally divergent. A recurrence policy never intervenes in 24 runs and reproduces fixed-rate trajectories exactly; each of four policies has 9 divergent cases. Ten figures distinguish pilot evidence, post-run diagnostics, exact algebra, and proposed mechanisms. The next experiments first verify a neural periodic regime, then measure its expression in outputs and task behavior, and finally test causal interventions against matched loss, recurrence, stability, and ordinary performance-map controls. Utility includes discovery costs and failed cases; it is not inferred from visual prominence.

AI assistance and editorial status. Generative-AI tools assisted with design, code, execution, local audits, source checking, and drafting. The human selected the question. Human shaping, artifact inspection, authorship, and final claim approval remain pending. The canonical publication disclosure will be applied at release only when true. No paid Pro review was requested or run.

1 The useful question behind a striking pattern

A shrimp structure is evidence that changing controls changes the long-run dynamics in an organized way. Its period, attraction strength, and bifurcation boundaries describe real properties of the update rule. The practical opportunity is to turn that organization into a decision: stay in a region, leave it, move to a more robust interior, or stop a computation whose answer is already stable.

A *shrimp* conventionally denotes a characteristic organization of periodic windows in a *control-parameter plane*. The associated periodic orbit lives in the evolving state space (Figure 1). Gallas described such domains for the Hénon map [3]. Da Silva and Rech report them in a three-neuron continuous-time Hopfield-type system with tanh activations [2]. Neural existence therefore has a specific precedent; usefulness for learning or reasoning remains the question here.

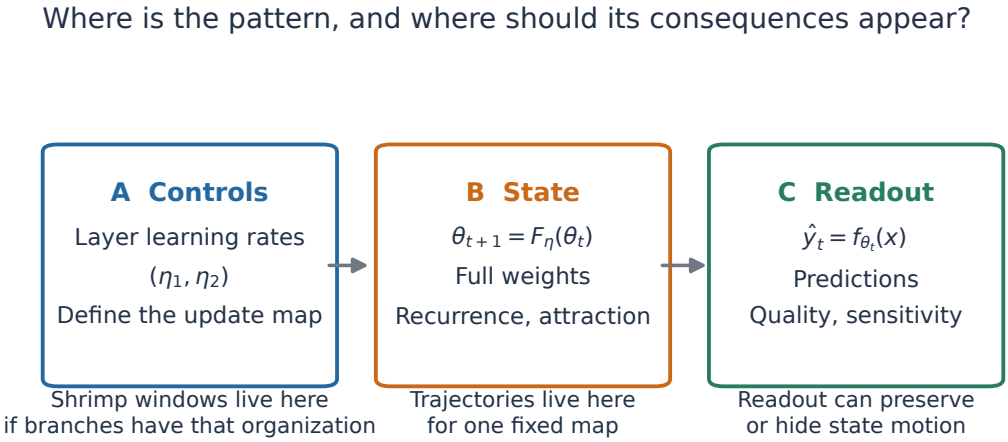


Figure 1: Schematic. Three spaces that answer different questions. A control-parameter map describes how an update rule changes across settings; a state-space trajectory describes motion at one setting; a task readout measures what that motion accomplishes. The diagram defines relationships and contains no detected neural structure. Learning rates are controls during deterministic training, while weights evolve; during recurrent inference, hidden states evolve under a fixed trained model.

Sohl-Dickstein's preprint and companion article map training success across hyperparameters [6, 7]. Lai and colleagues map transient dynamics over slices of initial latent states in recurrent reasoning models [5]. These observations motivate the search, but neither map as presented identifies the primitive periodic orbits and parameter bifurcations required here. Oscillatory optimization and period-doubling routes to chaos also have prior neural-training precedents [1, 4]. Our proposed contribution is a test of the *incremental operational value* of parameter-window geometry.

2 What must manifest, and what a readout can hide

2.1 A real internal signature need not change the answer

Let $z_{t+1} = F_\eta(z_t)$ be the complete autonomous state update, and let $q_t = R(z_t; x)$ be the output used by a task. A verified attracting period-p orbit is an internal manifestation: nearby trajectories return to the orbit and repeat after p steps. A shrimp organizes where that behavior exists as η varies. Neither statement requires the task readout to separate its phases.

If $R(z_0; x) = \cdots = R(z_{p-1}; x)$, the entire internal cycle is invisible to that readout. Distinct network states may implement the same prediction through symmetries, inactive units, or directions to which the output is insensitive. A decoded answer is still coarser: changing logits can retain the same argmax. Figure 2 makes the distinction explicit. An internally strong signal can therefore be harmless for accuracy while still affecting computation, noise response, or sensitivity to perturbations. It can also have no measurable effect on a chosen endpoint at the tested resolution. The experiment must determine which case occurs.

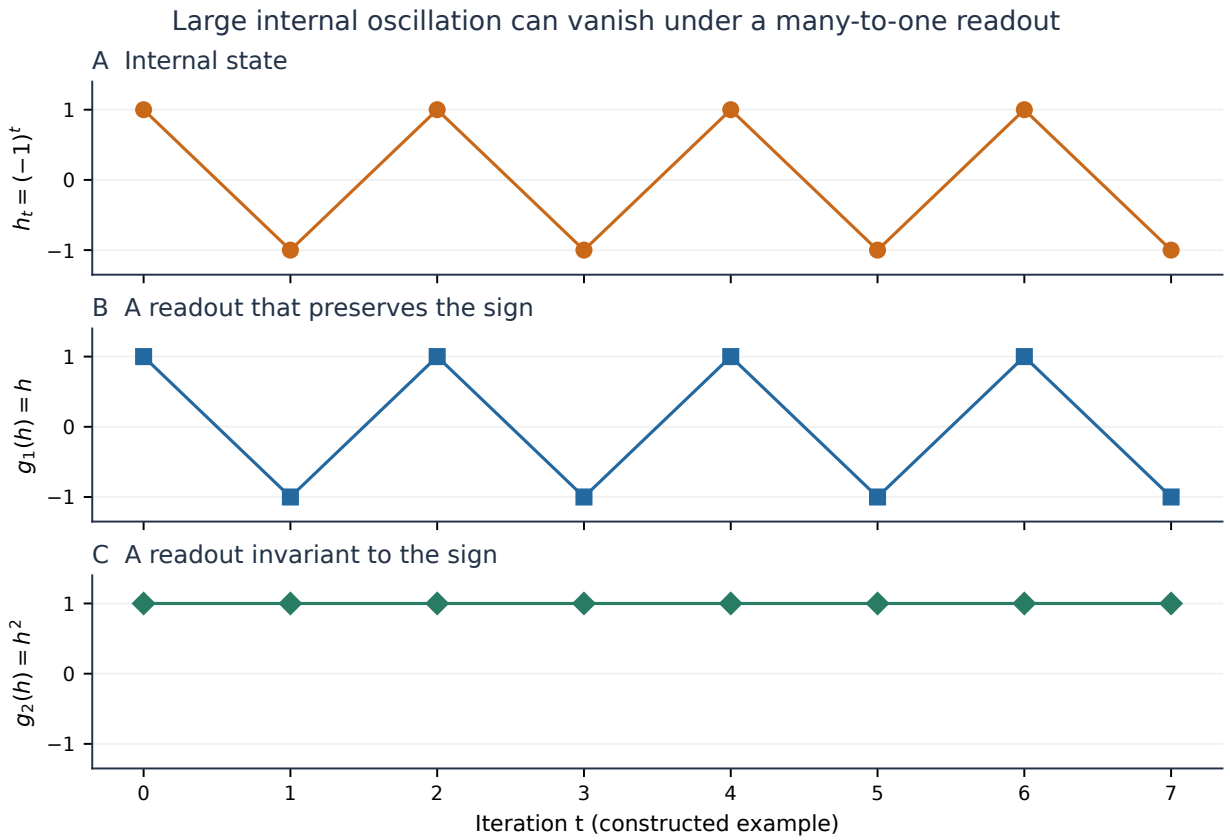


Figure 2: Algebraic illustration. (A) The constructed map $F(h) = -h$, initialized at $h_0 = 1$, alternates between $+1$ and -1. (B) The identity readout preserves that oscillation. (C) The squared readout hides it completely. This exact example makes no attraction, correctness, or shrimp claim: the two-step multiplier is $+1$. It demonstrates why a real internal signal need not survive a many-to-one readout.

For a verified orbit, define state and output amplitudes

$$A_z^2 = \frac{1}{p}\sum_{k=0}^{p-1} \|z_k - \bar{z}\|^2, \qquad A_q^2 = \frac{1}{p}\sum_{k=0}^{p-1} \|R(z_k; x) - \bar{q}\|^2. \tag{1}$$

The first detects state motion; the second measures its visibility through a specified readout. Their units differ, so neither their ratio nor a universal common threshold is assumed meaningful. Report phasewise gradients, task loss, answer disagreement, and predictive margins alongside them. These are proposed follow-up measurements, not measurements already made for a verified neural orbit.

2.2 Three mechanisms worth testing

Recovery and robustness. Attraction suggests a local ability to recover from perturbations. It does not guarantee a large basin or a correct output. Measure the probability of returning to the same orbit or answer under a frozen perturbation distribution, recovery time, and the change in task quality. Across a window, ask whether interior locations remain useful under larger control perturbations than locations near its boundary.

Intervention direction. A measured boundary supplies a possible direction and scale for changing controls. If an incorrect cycle wastes a budget, crossing that boundary may release it; if a useful

regime is fragile, moving inward may improve robustness. Compare either action with equal-size changes in random, tangential, reverse, and ordinary performance-informed directions. Leaving a stable cycle is an intervention, not an improvement until a task endpoint improves.

Stopping and aggregation. If predictions agree over a complete cycle and are correct, additional iterations may be unnecessary. If predictions differ across phases, phase averaging or phase selection may help or harm. Compare a cycle-aware rule with ordinary output agreement, residual, and stagnation rules, charging the cost of detecting a cycle. Test aggregation against matched numbers of evaluations and existing averaging controls. Stability alone provides no accuracy guarantee.

3 From recurrence to verified parameter structure

3.1 State and controls

For deterministic training on $\mathcal{D}$, the pilot uses

$$\theta_{t+1} = F_\eta(\theta_t) = \theta_t - D_\eta \nabla L_{\mathcal{D}}(\theta_t), \qquad D_\eta = \operatorname{diag}(\eta_1 I, \eta_2 I). \tag{2}$$

All weights and biases are state variables; positive layer learning rates are controls. For latent inference, a fixed model and problem instead define $h_{t+1} = G_\alpha(h_t; x)$. An autonomous description must include every evolving cache, auxiliary latent, momentum variable, and schedule phase.

Continuous gradient flow obeys

$$\dot{\theta} = -\nabla L(\theta), \qquad \frac{dL}{dt} = -\|\nabla L(\theta)\|^2. \tag{3}$$

It cannot support a nonconstant periodic orbit: integrating over a period forces the gradient to vanish throughout. Finite-step descent need not preserve monotonic loss, allowing the periodic-orbit search without guaranteeing a shrimp.

3.2 The verification ladder

A periodic point has $F_\eta^p(z) = z$ and no smaller positive period. Scalar loss recurrence is insufficient because different states can share the same loss. The pilot screens complete-state tails over repeated cycles, checks nonzero amplitude relative to numerical tolerance, rejects smaller periods, and independently checks orbit closure. For an orbit $z_0, \ldots, z_{p-1}$, its period Jacobian product is

$$M_p = DF_\eta(z_{p-1}) \cdots DF_\eta(z_0). \tag{4}$$

A spectral radius $\rho(M_p) < 1$, with a margin exceeding numerical error, establishes local attraction of an isolated orbit. It does not estimate basin size. Flat or symmetry-related directions may produce neutral multipliers; near-unit values require nonhyperbolic or unresolved status rather than discarded directions.

Periodic pixels alone do not establish shrimp geometry. A later stage must continue branches and locate saddle-node and period-doubling boundaries around connected isoperiodic regions. Longer horizons, changed precision, and multiple initial states assess long transients and multistability. Coordinate choices also matter: nonlinear axis transformations alter widths, normals, and apparent shape. Freeze meaningful coordinates such as log learning rates and distinguish coordinate-dependent distances from bifurcation organization.

4 The exploratory experiment and evidence record

The initial target is full-batch training of a tanh network with two inputs, three hidden units, one scalar output, and biases in both layers. All thirteen parameters evolve. The objective is half mean squared error plus $0.001\|\theta\|^2/2$, including biases in the penalty; predictive MSE excludes the penalty. A random tanh teacher defines independent standard-normal input splits with 24 training, 48 validation, and 96 test examples. Teacher metadata, splits, and initial states were prepared before the freeze, bound by manifest hashes, and verified on loading. This is inspired by the fractal-training construction, not an exact replication.

Result-free manifests specify a 65-cell Hénon control at 3,000 updates per cell and a 9×9 layer-learning-rate scan for seeds 101 and 102 at 2,000 updates per cell. Each seed defines both a task and its one initialization; these sources of variation are not separately estimated. Scans retain up to 512 tail states, screen periods through 16, and use float64. All requested cases remain in the census. A bounded, apparently aperiodic tail is not automatically chaos.

The policy comparison uses four task/initialization seeds, 201–204, and six initial rate pairs: 96 runs across fixed rates, predetermined decay, scalar stagnation-triggered decay, and recurrence-triggered decay. Each has at most 1,200 gradient updates and two interventions, each halving both rates. Scheduled decay acts at updates 400 and 800; adaptive rules inspect every 40 updates from update 200. Recurrence screens full weights without online orbit certification. Both adaptive detectors wait at least two check intervals after a rate change and use history only from the current constant-rate segment. Stagnation compares two loss windows from that segment.

Every generated state, pre-update regularized loss, realized learning rate, and trigger is retained. Test and validation outcomes do not trigger actions. The descriptive endpoints are terminal test MSE and divergence fraction; the report provides a policy mean only if all corresponding runs are finite. It provides no inferential p-values or confidence intervals. Equal maximum update counts do not imply equal total cost: divergent runs stop early, diagnostics consume resources, and wall time includes common instrumentation.

The initial runs used revision `bd3d733b51e9` after the full code, plan, and input revision was pushed to the private repository. Receipt verification checks hashes and the exact trial census. Empirical counts below are generated through `generated-pilot.tex`. Immutable attempts and descriptive reports remain under `evidence/`. The present visual revision adds derived figures and diagnostics; it does not create a new experiment or change the original classifications.

5 What the pilots actually show

5.1 The classical control finds periodic orbits

The Hénon control verifies attracting nontrivial cycles in 39 of 65 sampled cells (Figure 3). These are counts of cells, not distinct windows. Figure 4 connects a parameter-map classification to its state trajectory and local orbit check. This establishes that the instrument detects periodic orbits in the tested classical system. The sparse grid does not certify shrimp topology, and the positive control does not validate an unobserved neural orbit.

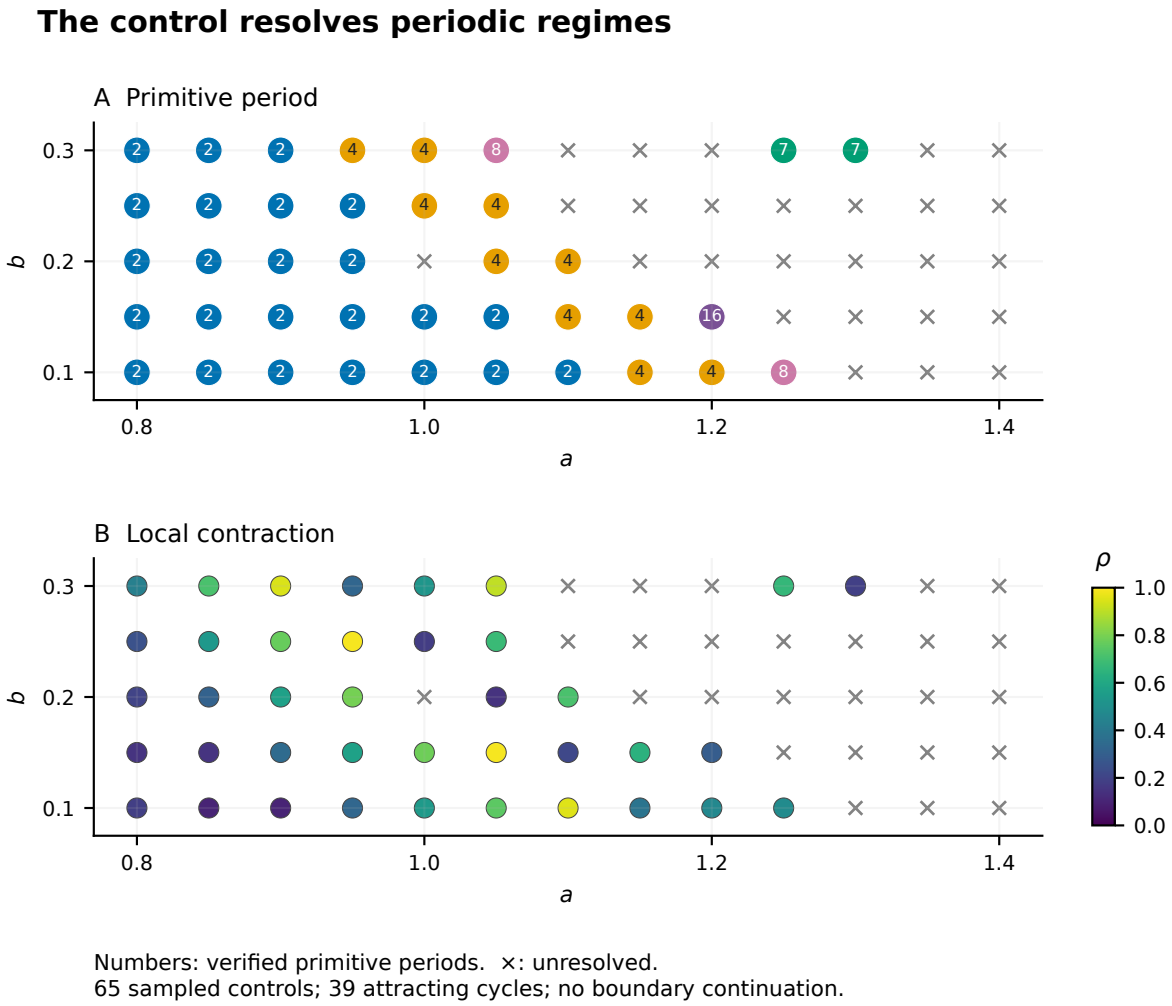


Figure 3: Observed. Hénon control across all 65 planned cells. The two panels show sampled primitive periods and the spectral radius for certified orbits. 39 cells contain numerically verified attracting nontrivial cycles. Radii cover each complete primitive cycle; differing periods prevent direct per-update comparisons. Values derive from frozen pilot evidence. A coarse map of periodic cells is a numerical positive control, not a certified reconstruction of shrimp boundaries.

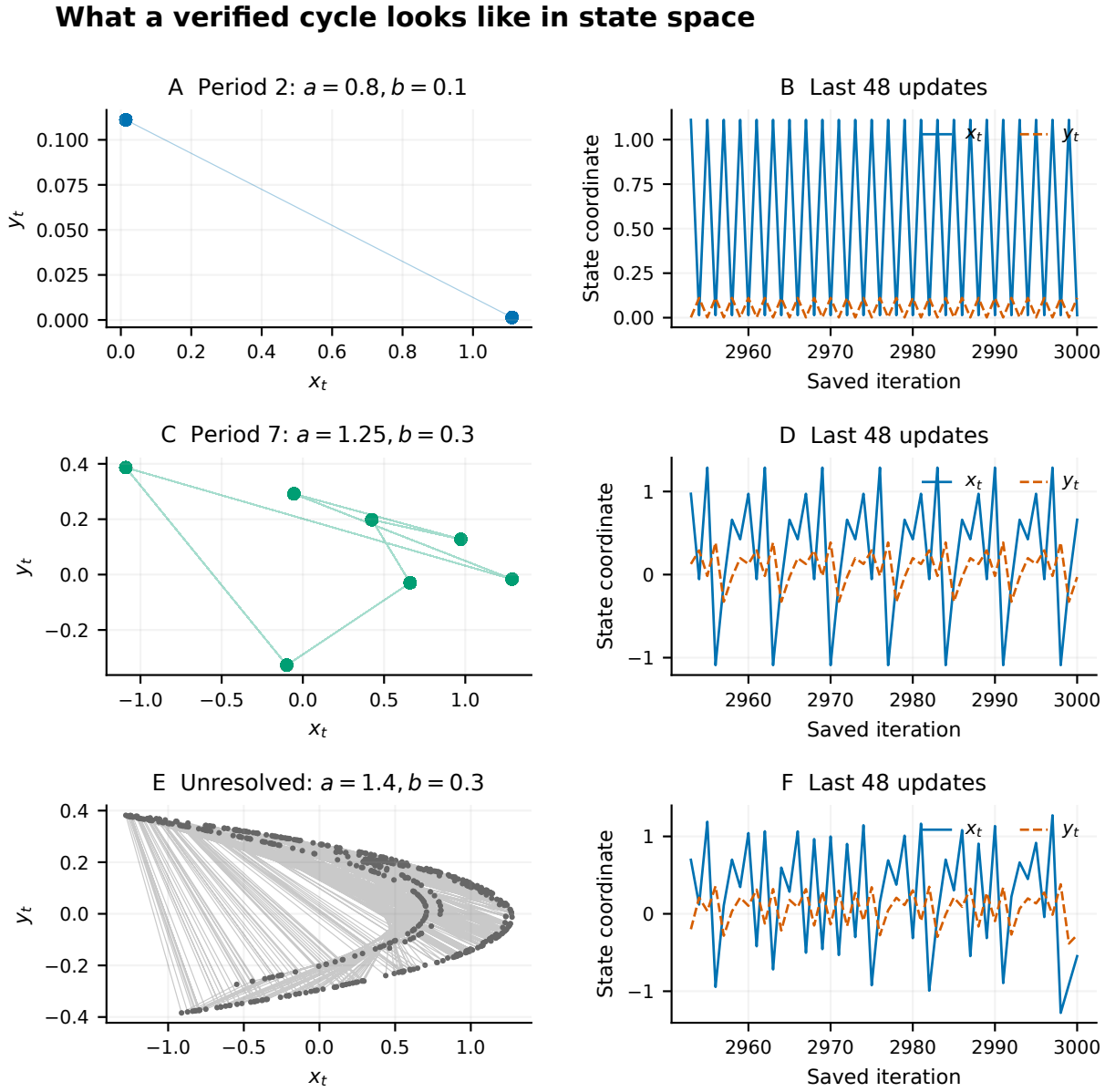


Figure 4: Observed. Representative Hénon trajectories connect the control map to motion in complete state space. Rows show period-two, period-seven, and unresolved cases; columns show phase-plane tails and coordinate time traces. Selection is post-run and descriptive, with the rule recorded alongside the figure. Connecting segments join discrete iterates. Only certified examples carry a periodic-orbit claim. These classical trajectories do not establish a neural result.

5.2 The neural scan separates divergence from unresolved dynamics

Among 162 neural cells, the pilot finds 0 verified attracting nontrivial cycles, 107 unresolved cases, and 55 operationally divergent cases (Figure 5). A finite grid and 2,000-update horizon do not establish global absence. They do show that this scan has not yet supplied the periodic regime required for a utility experiment.

Figure 6 examines the saved tails after the run. Small successive changes can reflect continuing drift. Comparing complete-weight updates, prediction changes, and loss changes helps decide where a successor search should spend effort. These post-run diagnostics preserve every original status and do not reclassify unresolved trajectories as chaos or as cycles.

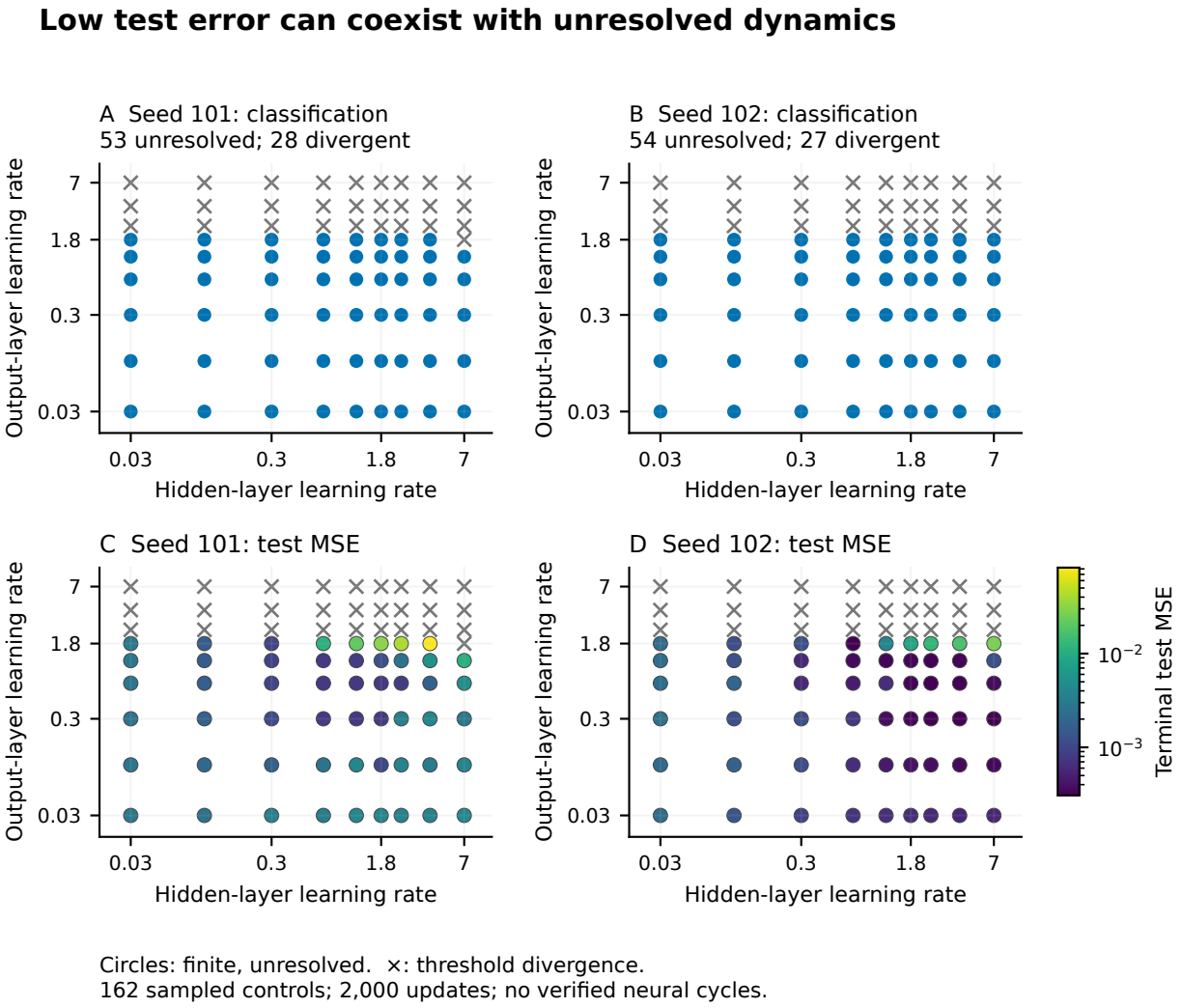


Figure 5: Observed. Both complete 9×9 neural learning-rate scans: task/initialization seeds 101 and 102 occupy separate columns, with classification above and test MSE below. The census retains all 162 cells: 0 verified attracting nontrivial cycles, 107 unresolved, and 55 operationally divergent. Maps display the actual sampled settings; visual boundaries are limited by grid resolution. Neither unresolved cells nor empty periodic classifications certify chaos or global absence of neural periodicity.

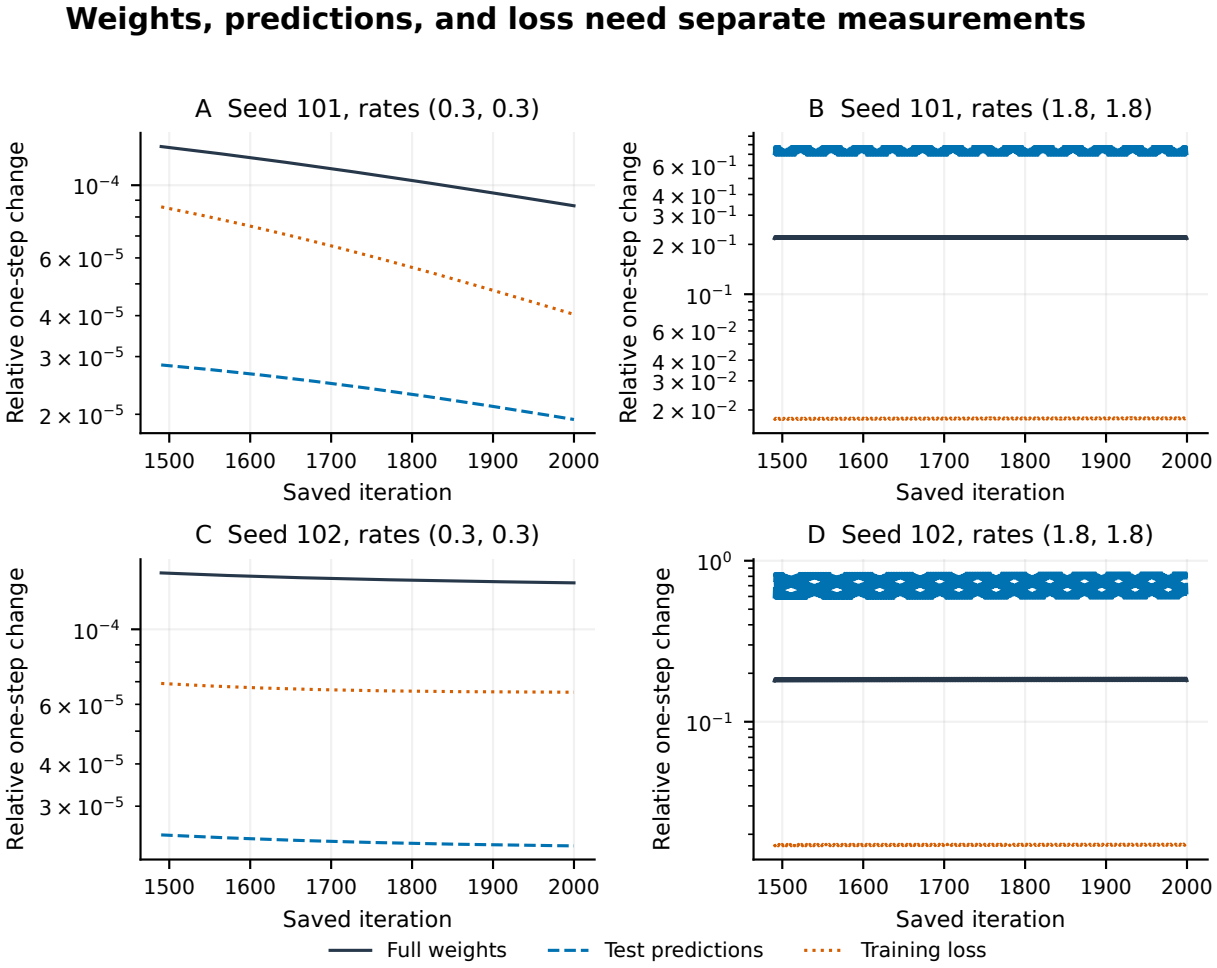


Figure 6: Post-run diagnostic. Relative successive changes in full weights, predictions, and loss, each using its own denominator, for seeds 101 and 102 at equal layer rates $(0.3, 0.3)$ and $(1.8, 1.8)$. These four displayed cases and the diagnostics were chosen after outcomes were available. They guide a separately specified successor rather than provide a prospective endpoint. Original statuses remain unchanged; slow motion and small successive changes do not certify a nontrivial cycle.

5.3 The first recurrence controller never changes a trajectory

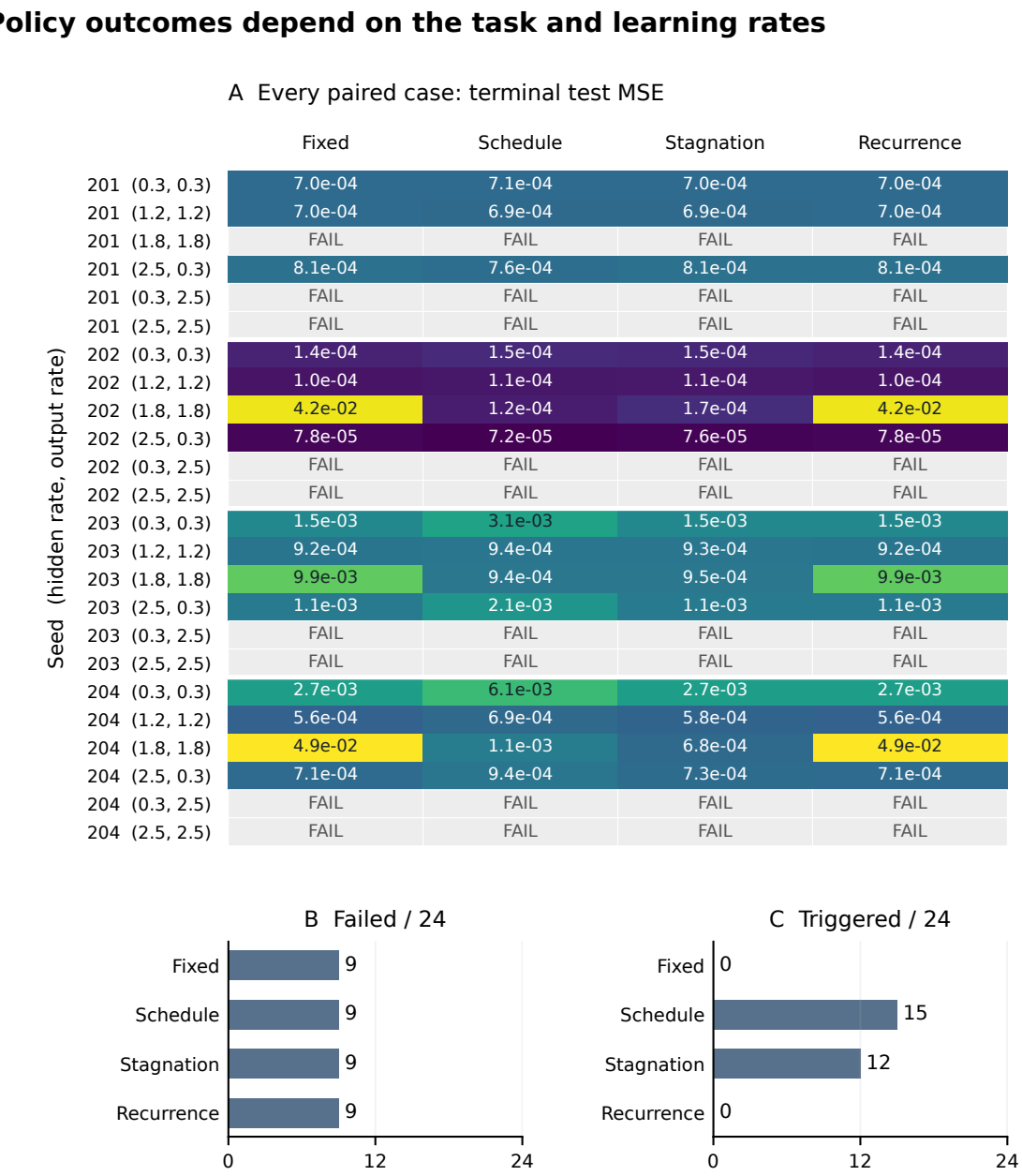


Figure 7: Observed. (A) Complete terminal-MSE matrix, (B) failure counts, and (C) counts of runs with an intervention for four policies across 24 shared task–rate pairs each. Every policy retains 9 divergent cases. The recurrence detector never acts. Successful-case outcomes, where shown, are descriptive and conditional on completion; they do not replace the full failure census or the prespecified decision to withhold an all-case mean when divergence occurs.

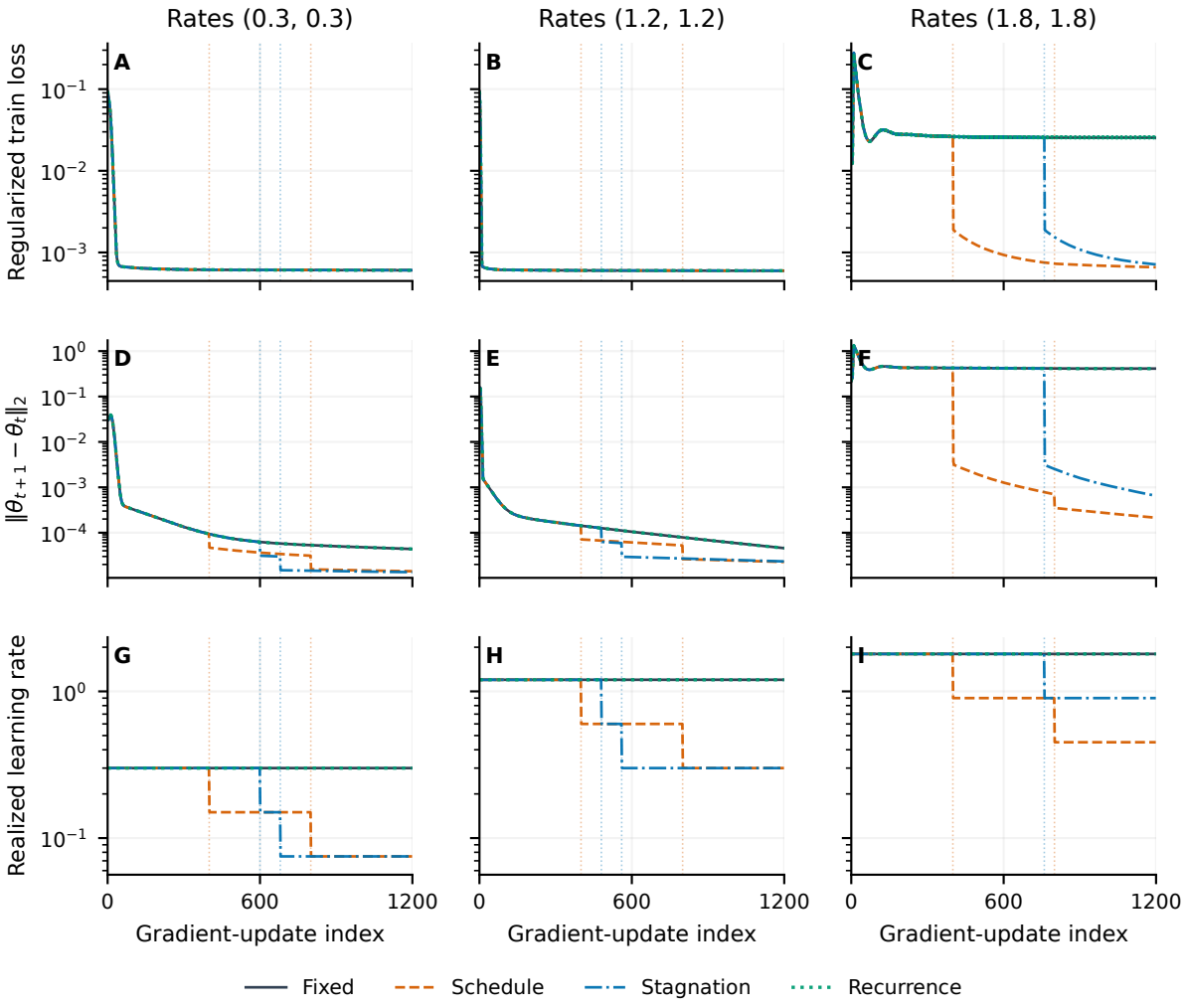


Figure 8: Observed. Saved policy trajectories for seed 202 at equal layer rates 0.3, 1.2, and 1.8 occupy separate columns; rows show regularized loss, update norm, and realized learning rate. These are the first three manifest-order pairs that remain finite under every policy, chosen after the run for descriptive comparison. Fixed-rate and recurrence-policy arrays agree exactly over all 24 pairs, beyond the displayed examples. Different decay policies can change a trajectory, but these illustrative paths do not establish a held-out benefit, a speedup, or a causal effect of shrimp geometry.

The policy comparison includes 96 runs; each policy has 9 divergent cases among 24 task–rate pairs (Figure 7). The recurrence policy triggers in 0 runs. An independent comparison finds exactly equal saved state, loss, and learning-rate arrays for fixed-rate and recurrence policies on every pair, comparing NaNs at matching positions. Statuses, completed updates, and terminal predictive MSE also match.

Figure 8 shows how those outcome counts arise along representative saved paths. Here the recurrence detector adds observation overhead without changing dynamics. This is a non-intervening pilot, not an equivalence test of effective control methods. It contains no geometry-based intervention and cannot estimate geometry-specific benefit.

Every policy retains divergent cases, so the descriptive report withholds a successful-only mean test MSE. There is no significance or net-compute claim. Runs used local CPU resources, with zero recorded paid compute and paid review spend; local wall time is not a total economic-cost estimate. The immediate scientific need is a reproducible neural periodic regime, followed by a measurement of what it does to the task.

6 An exact neural toy: a cycle changes predictions

A scalar example makes the manifestation question concrete without a new numerical experiment. Consider a two-layer linear network with its two scalar weights tied, $\widehat{y} = \theta^2 x$, trained on $(x, y) = (1, 1)$ using $L(\theta) = \frac{1}{2}(\theta^2 - 1)^2$. Gradient descent gives

$$F_\eta(\theta) = (1 + 2\eta)\theta - 2\eta\theta^3. \tag{5}$$

This is a deliberately simple neural parameterization. Its exact cycles are not a shrimp, a result for the tanh pilot, or evidence of generalization.

Set $r = 1/(2\eta)$. For $\eta > 1/2$, a positive period-two orbit has

$$u, v = \frac{\sqrt{1+3r} \mp \sqrt{1-r}}{2}, \qquad uv = r, \qquad (u+v)^2 = 1 + 3r. \tag{6}$$

Substitution gives $F_\eta(u) = v$ and $F_\eta(v) = u$. Its two-step multiplier is

$$\mu = F'_\eta(u)F'_\eta(v) = 9 - 2(1 + 2\eta)^2, \tag{7}$$

so the nontrivial cycle attracts locally for $1/2 < \eta < (\sqrt{5} - 1)/2$. The endpoints have $\mu = 1$ and $\mu = -1$, respectively, and are excluded from that attraction claim.

The outputs u^2 and v^2 differ. Their mean is $(1 + r)/2 < 1$, making the task consequence explicit. Three squared-error quantities on this one sample are

$$\mathrm{MSE}_{\text{random phase}} = (1 - r^2)/2, \tag{8}$$

$$\mathrm{MSE}_{\text{mean prediction}} = (1 - r)^2/4, \tag{9}$$

$$\mathrm{MSE}_{\text{mean weight}} = 9(1 - r)^2/16. \tag{10}$$

The first averages squared errors over uniformly sampled cycle phases; the second squares the error after averaging predictions; the third evaluates the network at $(u + v)/2$. Thus the same internal cycle supplies both a visible output oscillation and distinct aggregation choices (Figure 9). Prediction averaging improves on random-phase evaluation here, as the squared-loss variance decomposition predicts. It does not eliminate bias and does not beat the zero-error solutions $\theta = \pm 1$, which are locally attracting for $0 < \eta < 1/2$.

This calculation establishes a concrete consequence of neural periodicity and a conditional aggregation benefit. It establishes neither a net compute gain nor additional value from shrimp geometry. Its proper experimental role is a transparent control for measuring phasewise outputs and separating prediction averaging from weight averaging.

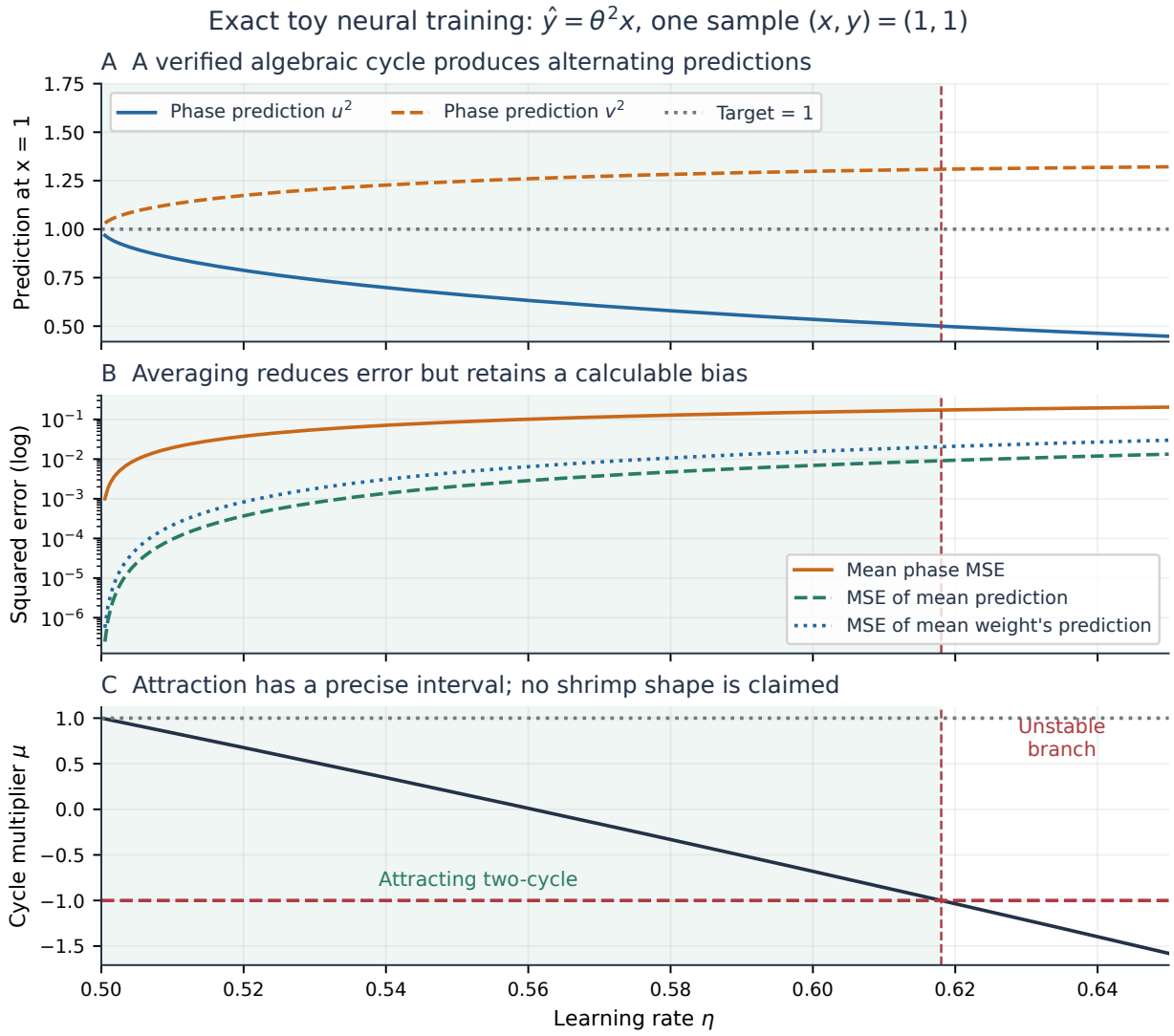


Figure 9: Exact analytic illustration. (A) Phase predictions, (B) three squared-error readouts, and (C) the local multiplier for the tied scalar network $\widehat{y} = \theta^2 x$ on the single training sample $(1, 1)$. Green shading marks local attraction; the continuation beyond its right boundary is unstable. Averaging predictions lowers error relative to uniformly selecting a phase, but retains bias. Zero-error fixed points remain available at other stable rates. Curves evaluate equations in the text, not a new pilot, a parameter sweep with fitted outcomes, or a verified shrimp.

7 A sequence of experiments with decisions at every stage

Figure 10 ties each stage to the decision its evidence supports. Follow-up discovery and controller development remain exploratory. A confirmatory evaluation requires a new frozen machine plan binding samples, thresholds, features, interventions, budgets, and analysis before held-out outcomes are available.

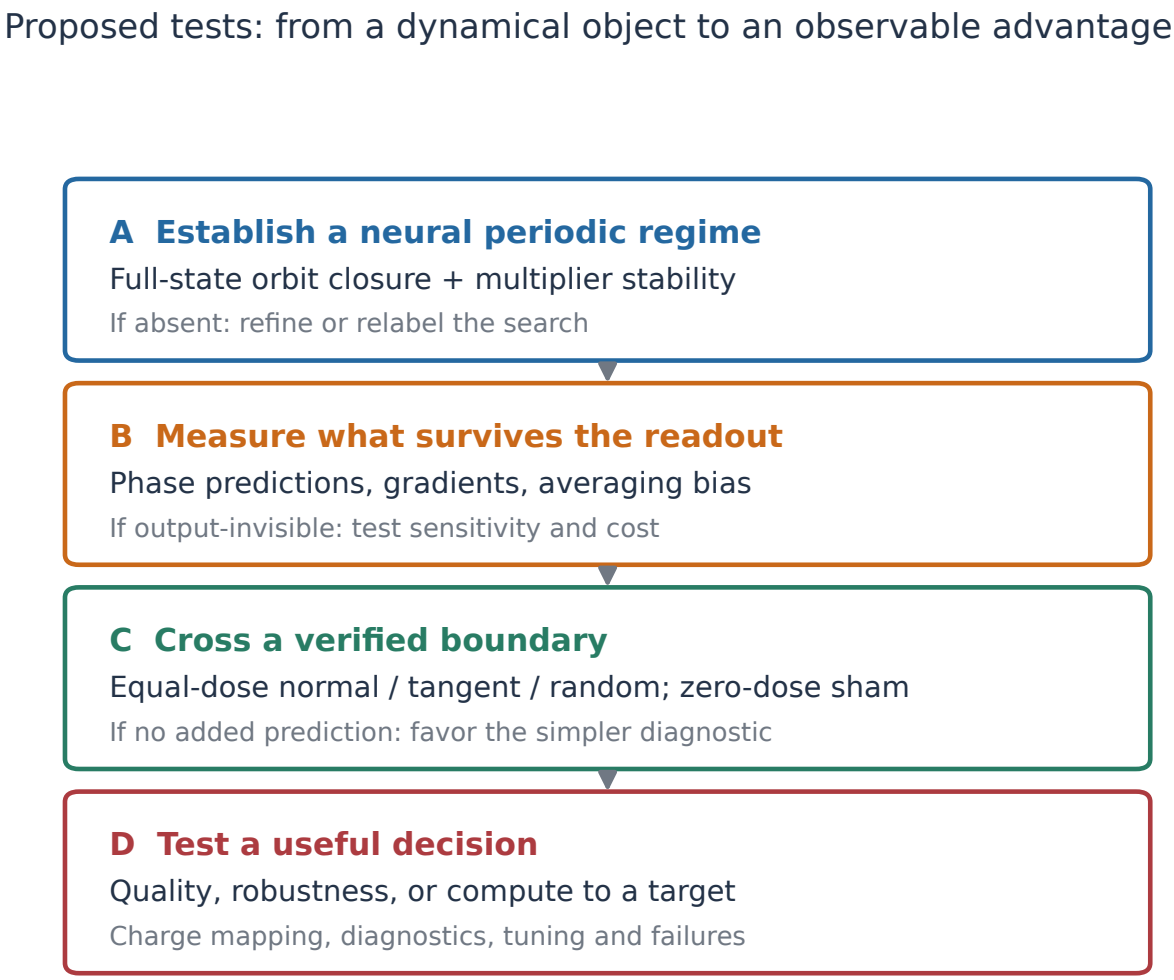


Figure 10: Proposed. (A) Establish a verified neural periodic regime; (B) measure what reaches the task readout; (C) test actions across verified parameter boundaries; (D) evaluate quality, robustness, and total cost. Each stage produces a measurement and a decision, with advancement depending on preceding evidence. No benefit, neural shrimp, or completed successor experiment is represented by this diagram.

7.1 Establish a regime before asking it to improve a model

First use the analytic scalar cycle to check orbit and output measurements, then reproduce a pinned published small-neural-system periodic regime with its equations and parameter conventions independently checked. Separately specify a targeted successor to the training scan: longer horizons, parameter refinement, and complete-state orbit verification. Preserve the empty scan and every

failed candidate. A known neural positive control demonstrates instrument capability; it is not a new discovery or evidence that the current training system has a shrimp.

Once a neural periodic orbit is verified, continue its branch and relevant bifurcations across two controls. Retain stable cycles that lack shrimp geometry as a comparison group. This separates value supplied by periodicity from any value supplied by the window's organization. Boundary uncertainty must be propagated into any proposed boundary direction or width; a coarse visual outline is insufficient.

7.2 Map consequences at the same parameter settings

For each verified regime, pair the dynamical map with a task-observable map: predictive loss or exact correctness, phasewise output disagreement, recovery probability and time under perturbations, and compute to a declared quality threshold. Include matched nearby nonperiodic settings, fixed points, and ordinary periodic windows. The primary question is whether the same boundaries organize changes in these observables, and whether those changes survive changes in task and initialization.

Measure complete-state motion and output motion separately. Include a readout-insensitive or symmetry-controlled case to test whether the analysis mistakenly converts internal structure into a claimed output effect. If a clear state-space signal has no task-visible effect under the tested interventions, report that as a mechanism result. It bounds the effect detected through that readout on the tested inputs, horizon, and precision. An exact invisibility claim requires a separate invariance argument, such as a demonstrated symmetry.

7.3 Branch checkpoints to test intervention effects

At common checkpoints, apply a frozen set of equal-size control changes. Compare boundary-informed directions with random, tangential, reverse, shuffled-map, and ordinary locally sampled performance-informed directions in prespecified log-learning-rate coordinates. Include a zero-dose sham with identical diagnostic overhead; it controls observation and timing, not intervention magnitude. Match intervention doses, opportunities, state access, and tuning budgets. Test both escape from harmful cycles and movement toward useful robust regimes. Measure post-intervention task utility, recovery, and failures, rather than treating boundary crossing as success.

A predictive comparison uses identical fitted models and tuning budgets. Baseline features include recent loss, progress, gradient and update norms, recurrence residual, estimated period, cycle amplitude, and local stability, including available orbit multipliers. The augmented model adds boundary distance, local width, boundary direction, and measured window organization. Period information is shared so its value cannot be misattributed to geometry. Labels are future task utility after a specified action, not recurrence itself. Hold out whole windows or spatial blocks as well as tasks, so local interpolation does not masquerade as transfer.

7.4 Demand a practical advantage over simpler controls

Required controller baselines are a predetermined schedule, stagnation, recurrence, local stability, and ordinary performance maps with matched scan costs. Include output agreement for stopping and matched averaging procedures for phase aggregation. An inactive detector, as in the current pilot, cannot answer the relative-effectiveness question.

The proposed primary endpoint is paired error on untouched evaluation/test data at a fixed total resource budget. These data are separate from any training or development-validation data used for map selection, controller tuning, or online actions. A complementary endpoint is compute to a quality threshold, with unsolved cases budget-censored. Reasoning experiments require exact

correctness at equal total inference cost. The exact aggregation, practical-effect threshold, divergence handling, cluster bootstrap, primary contrast, and multiplicity rule must be frozen before evaluation. A synthetic teacher or data-generating problem is the independent task unit; pixels and time points are not independent replications. Keep seed variation within tasks.

Charge map construction, tuning, failed probes, Jacobians, state storage, transfers, and extra validation. Report cold-start and explicitly amortized costs for a declared number of future runs. Reused experimental prefixes are charged to every hypothetical deployed policy. Report the unselected prevalence of harmful regimes: a high rescue rate in a deliberately enriched set of traps does not establish population-level savings.

8 A controlled extension to latent reasoning

Stable repetition and transient delay call for different interventions. The Lai et al. preprint uses decoded settling, discusses transient structure, and describes capped or inconsistent slices and slim fractals in its appendices [5]. Our extension would retain failures and measure both decoded correctness and complete-state recurrence.

Positive relaxation offers an interpretable intervention:

$$G_\alpha(h) = (1-\alpha)h + \alpha F(h), \qquad \alpha > 0. \tag{11}$$

It preserves the fixed-point equation: $G_\alpha(h) = h$ exactly when $F(h) = h$. At a fixed point, an eigenvalue λ of DF becomes

$$\mu = 1 - \alpha + \alpha\lambda. \tag{12}$$

Thus positive relaxation cannot stabilize a real $\lambda > 1$ direction, although it can change oscillatory stability. This local algebra does not guarantee global correctness or faster convergence; relaxation can change basins, transients, and nontrivial cycles.

A two-control experiment could use relaxation and momentum, including the previous iterate in the full state. Feedback gain is another control, but generally changes the fixed-point equation and therefore the solver itself. Compare damping, restarts, and geometry-informed changes against residual, output-agreement, and stagnation policies on untouched tasks at matched total inference budgets. Distinguish removing internal motion from rescuing an incorrect answer.

9 Interpretation and limits

The most concrete potential contribution of shrimp geometry is an intervention map: where a desired dynamic persists, which direction changes it, and how much perturbation it tolerates. Three operational outcomes would justify pursuing it: lower error at a fixed total cost, more reliable recovery under disturbances, or earlier correct stopping. A beautiful atlas alone establishes none of them.

The right action depends on the task. An incorrect attracting cycle is a candidate trap. A correct, output-stable cycle may admit earlier stopping. A useful but fragile regime may benefit from an inward move. A cycle invisible to the chosen readout may need no intervention. For every case, the relevant comparison is with a simpler diagnostic given the same information and resources.

Several negative outcomes would still answer the question. If ordinary recurrence matches an active geometry method, a cheaper observable may suffice. If mapping costs erase the per-run gain, there is no net advantage at that deployment scale. If a change improves training loss but harms validation, both must be reported. If an internally strong pattern is removed by the readout, the

study should identify that mechanism without claiming a hidden task benefit. The current pilot establishes neither a neural shrimp nor a control advantage, and limits its negative evidence to the tested settings, horizons, precision, and initialization choices.

Reproducibility and research status

The private repository contains executable pilots, frozen inputs and configurations, immutable raw attempts, descriptive reports, figure-generation code and provenance, a research plan, a claim ledger, and reference records. Figures are rebuilt by `tools/build_evidence_figures.py` and `tools/build_concept_figures.py`. Each `paper/figures/*.receipt.json` records its sources, hashes, selection or construction, and plotted values where applicable. The gallery in `docs/visual-revision.md` maps figures to their evidence and claim scope. The initial program is exploratory and has no registry record. Private Git history supports internal chronology, not public prospective verification. Confirmatory work requires a fresh frozen design, retained failures and amendments, full cost receipts, and outcome-linked claims. Human editorial review and release decisions remain pending; routine local audits do not require paid Pro review.

References

[1] Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations*, 2021. URL https://iclr.cc/virtual/2021/poster/2577.

[2] Angela da Silva and Paulo C. Rech. On bifurcations along the spiral organization of the periodicity in a Hopfield neural network. *Communications in Advanced Mathematical Sciences*, 5 (2):92–98, 2022. doi: 10.33434/cams.1064713. URL https://dergipark.org.tr/en/pub/cams/article/1064713.

[3] Jason A. C. Gallas. Structure of the parameter space of the Hénon map. *Physical Review Letters*, 70(18):2714–2717, 1993. doi: 10.1103/PhysRevLett.70.2714. URL https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.70.2714.

[4] Dayal Singh Kalra, Tianyu He, and Maissam Barkeshli. Universal sharpness dynamics in neural network training: Fixed point analysis, edge of stability, and route to chaos. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=VZN0irKnl0. Camera-ready text inspected as arXiv:2311.02076v2, 13 February 2025; accessed 8 September 2026.

[5] Jeffrey Lai, Anthony Bao, John Quinn, and William Gilpin. Fractal basins trap latent reasoning, 2026. URL https://arxiv.org/abs/2609.04963v1. Preprint, version 1, 4 September 2026; accessed 8 September 2026.

[6] Jascha Sohl-Dickstein. The boundary of neural network trainability is fractal, 2024. URL https://arxiv.org/abs/2402.06184v1. Preprint, version 1, 9 February 2024; accessed 8 September 2026.

[7] Jascha Sohl-Dickstein. Neural network training makes beautiful fractals. Author's blog, 12 February, 2024. URL https://sohl-dickstein.github.io/2024/02/12/fractal.html. Accessed 8 September 2026.